Approved targeted diagnostics

PE6203 Group Project 4 | Addendum after explicit approval | 31 October 2026

Four prespecified trajectories were executed once each, with five judge ratings per trajectory. This addendum supplements the frozen 27-cell benchmark without changing its scores or strategy ranking.

Condition	Stage	TF	PE	Calls / errors	End status
U: missing fact	Late	10.0	5.8	27 / 9	Output truncated
U: complete fact	Late	10.0	5.6	5 / 1	Completed
O: rule reverted	Middle	9.8	10.0	4 / 0	Completed
O: rule retained	Middle	10.0	7.8	22 / 2	Output truncated

Findings and their scope

Under-update mechanism: an injected omission produces eight calls without the newly required allow_variables parameter; all eight are rejected. The complete snapshot has zero such rejections. This confirms a local mechanism under controlled fault injection, not a natural failure of original Selective Patch.

Extra-rule comparison: retaining one historical evaluate rule coincides with more calls, lower PE and truncation. The rollback also has slightly lower TF. With one stochastic execution per arm, this is suggestive evidence, not proof of general harm or a superior strategy.

Cost and stopping

Total recorded API cost: **$0.04600192**. There are no unknown request charges in this cohort. Execution and judging took **160.707 seconds (2.68 minutes)**, excluding preparation, analysis and packaging. No updater-model calls were made.

All four approved trajectories are terminal. No repetitions, new candidates, Netlify deployment or larger matrix were started. ECS is not defined for this single-stage two-condition diagnostic.

Evidence: audit.json; plan.json; raw/cells/{condition}/run/{raw_trace.json,EVAL/133.json,result.json}

U: missing parameter knowledge

Exact intervention

Task 133, Late. The control uses the exact archived main Selective Patch snapshot. The injected arm removes only parameters.properties.allow_variables and its entry in parameters.required for math-mcp::validate_syntax. All other knowledge and the live tool schema stay identical. This deliberately bypasses the original updater's correct output.

Observed failure and recovery

The first injected validation call uses **2 + 3 * (4 - 1) / 2** without allow_variables. The server returns: validate_syntax requires allow_variables to be explicitly provided as a boolean. All eight validation calls in rounds 1-3 omit the field and receive this rejection. The agent later switches to evaluate, makes 17 evaluation calls in total, and reaches the output limit.

The control supplies the field in both validation calls. Its five calls include a separate batch_evaluate call that returns an upstream implementation error: this.toolHandlers.handleBatchEvaluate is not a function. Its agent loop finishes normally.

Measure	Injected omission	Complete snapshot
Missing-field rejections	8	0
Calls / agent rounds	27 / 7	5 / 3
PE ratings	5, 6, 6, 5, 7	5, 5, 5, 5, 8
Mean TF / PE	10.0 / 5.8	10.0 / 5.6

Judge error and causal limit

Four control judge rationales incorrectly say batch_evaluate is outside the available tools. It appears in the knowledge, live metadata and model-facing tool list. The implementation does fail. Whether distinct validation/evaluation checks are redundant is also debatable. Every original rating remains unchanged.

Confirmed: absent arguments directly cause the eight local rejections. **Not established:** a natural Patch under-update, prevalence, or a PE benefit from the correction. PE is 0.2 lower in the control despite fewer errors and normal completion.

Evidence: exact-knowledge-diffs.json; under_omitted/run/events.jsonl; under_complete/run/{knowledge.json,environment-metadata.json,raw_trace.json,judge-attempts/}

O: one-rule rollback and limits

Exact rule under test

Task 133, Middle. One arm replays the historical linked_notes snapshot. The rollback changes only math-mcp::evaluate.usage_rules to []. Three other archived guideline-rule changes remain identical.

validate_syntax now accepts optional detailed and allow_variables flags; evaluate's validate_only remains a separate syntax-only path, so choose based on whether evaluation is also wanted.

Measure	Rule retained	Rule reverted
All calls / tool errors	22 / 2	4 / 0
validate_syntax / evaluate	9 / 11	2 / 1
evaluate in later rounds than first validation	10	0
Mean TF / PE	10.0 / 7.8	9.8 / 10.0
End status	Output truncated	Completed

The retained-rule run adds distinct security, arithmetic and numerical probes; not every extra call is necessarily redundant. Its two evaluation errors concern unsupported lambda syntax and undefined variables. The rollback performs four calls in its first round and then answers.

Suggestive: a single-field comparison is stronger than the historical four-field comparison, but one execution per arm and no provider seed leave stochastic variation unresolved. evaluate is related to the validator, so unchanged metadata does not establish unrelated knowledge. This is a historical variant, not the main method.

Fairness, preservation and course status

Archived runtime hashes match the main run: Python 3.12.12, Node v24.13.0 and unchanged Middle/Late catalogs. Settings remain DeepSeek v4.1 Flash through OpenRouter, temperature 0, 4096 tokens, 12 rounds, 900 seconds, and the same retry and five-slot judge protocol. All 20 ratings were independently recalculated.

All 10,959 previously packaged evidence files retain their hashes. The six negative description-rollback diagnostics and main PE 9.1111 remain intact. The strict gap for natural original-Patch under-update and genuinely unrelated over-update failures remains. Course acceptance of a controlled fault case needs justification; no more experiments are proposed automatically.

Evidence: driver.py; audit.py; plan.json; exact-knowledge-diffs.json; audit.json; package-verification.json